INNOMATICS®
RESEARCH LABS

INNOVATION. AUTOMATION. ANALYTICS

PRESENTATION

Inheritance and Abstraction in OOPS

Presented by:
Madhu sudan mal

About me

- **BBA(Bachelor of Business administration)**
- linkedin.com/in/madhu-sudan-mal-67b023316
- https://github.com/madhumal681-lab

Agenda

1. Concept of Inheritence
2. Types of Inheritence
3. Concept of Abstraction
4. Difference between Abstraction and Inheritence
5. Real World Examples
6. Conclusion

What is Inheritance?

Inheritance is an OOP(Object-Oriented Programming) concept where one class (child/subclass) acquires the properties and methods of another class (parent/superclass).

Advantages:

- **Code reuseability**
- **Easy maintenance**
- **Reduced redundancy**

Example: Parent class:Animal → methods:eats(), sleep()
Child class:Dog → inherits from Animal + methods: barks()

Types of Inheritence

Types include:

1. **Single Inheritence**

   One Child inherits from one parent

2. **Multiple Inheritence**

   One Child inherits from multiple parent

3. **Multilevel Inheritence**

   Inheritance in a chain(**A**→**B**→**C**)

4. **Hierarchical Inheritence**

   Multiple childern inherits from one parent

5. **Hybrid Inheritence**

   Combination of different types

Single Inheritance

Defination: Single Inheritence occurs when one child class inherits from one parent class.

```
class Animal:
    def speak(self):
        print('Animal ,makes sound')
class dog(Animal):
    def bark(self):
        print('Dog barks')
d = dog()
d.speak()
d.bark()
```

```
Animal ,makes sound
Dog barks
```

Multiple Inheritence

Defination: Multiple inheritance occurs when one child inherits from multiple parent.

```
class Father:
    def skills1(self):
        print('gardening')
class Mother:
    def skills2(self):
        print('Cooking')
class child(Father,Mother):
    pass
c=child()
c.skills1()
c.skills2()
```

```
gardening
Cooking
```

Multilevel Inherritence

Defination: Multilevel inheritance occurs when a class inherits from class which itself inherits from another class.

```
class Grandparent:
    def show1(self):
        print('Grandparent')
class parent(Grandparent):
    def show2(self):
        print('parent')
class Child(parent):
    def show3(self):
        print('child')
c = Child()
c.show1()
c. show2()
```

```
Grandparent
parent
```

Hierarchical Inheritance

Defination: Herarichial Inheritance occurs when multiple child classes inherits from one parent class.

```
class Animal:
    def eat(self):
        print('Eating')
class Dog(Animal):
    pass
class Cat(Animal):
    pass
d = Dog()
c = Cat()

d.eat()
c.eat()
```

```
Eating
Eating
```

Hybrid Inheritance

Defination: Hybrid inheritance is a Combination of two or more type of inheritance.

```
class A:
    def showA(self):
        print('class A')
class B:
    def showB(self):
        print('class B')
class C(A):
    def showC(self):
        print('class C')
class D(B,C):
    pass

d=D()
d.showA()
d.showB()
d.showC()
```

```
class A
class B
class C
```

What is Abstraction?

Abstraction is an OOP concept that hides unnecessary implementation details and shows only the essential features of an object.
It focuses on what to do rather than how to do it.

Example: using a car without knowing how the engine works

Benefits:

- **Reduce complexity**
- **Improve code readability**
- **Enhance security**

Difference Between Inheritance and Abstraction

Inheritence	Abstraction
● Reuse code from another class ● Parent-child relationship ● Improves code reuse	● Hides implementation details ● Focus on essential features ● Simplifies system design

Real-World Example for Inheritence

A Car is a type of Vehicle.
All Vehicle can:

- **Start**

A Car can also:

- **Drive**

So Car inherits properties from Vehicle

Python code:

```
class Vehicles:
    def start(self):
        print('Vehicles Started')

class car(Vehicles):
    def drive(self):
        print('Car is driving')

c= car()
c.start()
c.drive()
```

Output:

```
Vehicles Started
Car is driving
```

Real-World Example for Abstraction

We create an abstract class shape.
Different shapes implement their own area() method.
Example:

- **Circle**
- **square**

Python code:

```
from abc import ABC, abstractmethod
class shapes(ABC):
    @abstractmethod
    def area(self):
        pass
class circle(shapes):
    def area(self):
        print('area of circle')
class square(shapes):
    def area(self):
        print('area of square')

c = circle()
c.area()
```

Output:

```
area of circle
```

Conclusion

Inheritance and Abstraction are important concepts in Objects-Oriented Programming.

They help developers:

- **Reuse code**
- **Reduce complexity**
- **Improve program structure and efficiency**

THANK YOU